

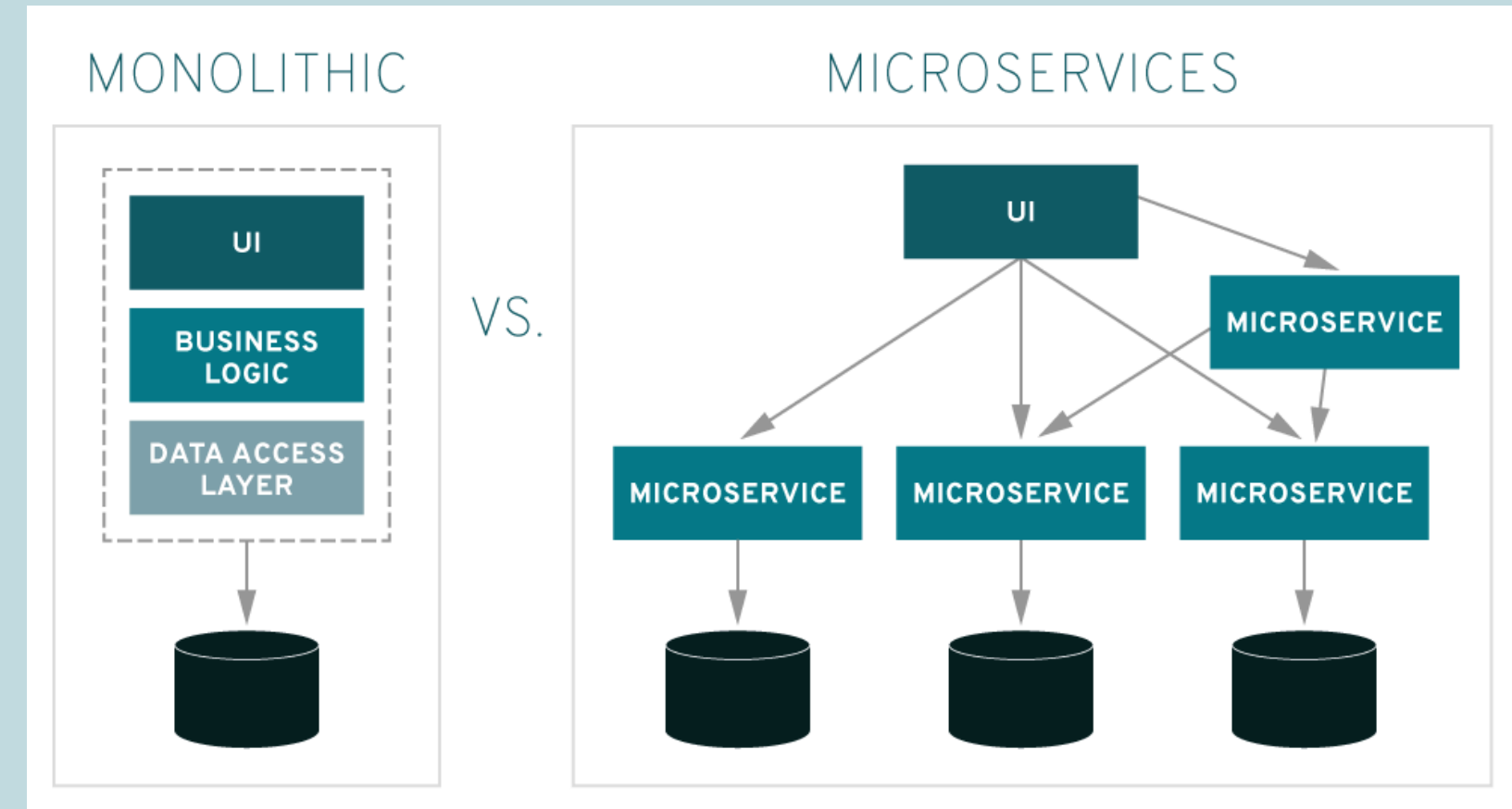
EXPLORACIÓN DE ESTILOS ARQUITECTÓNICOS Y STACK TECNOLÓGICO

Grupo 4

ESTILO ARQUITECTÓNICO: MONOLITO

ES una sola aplicación, un solo código y un solo despliegue.

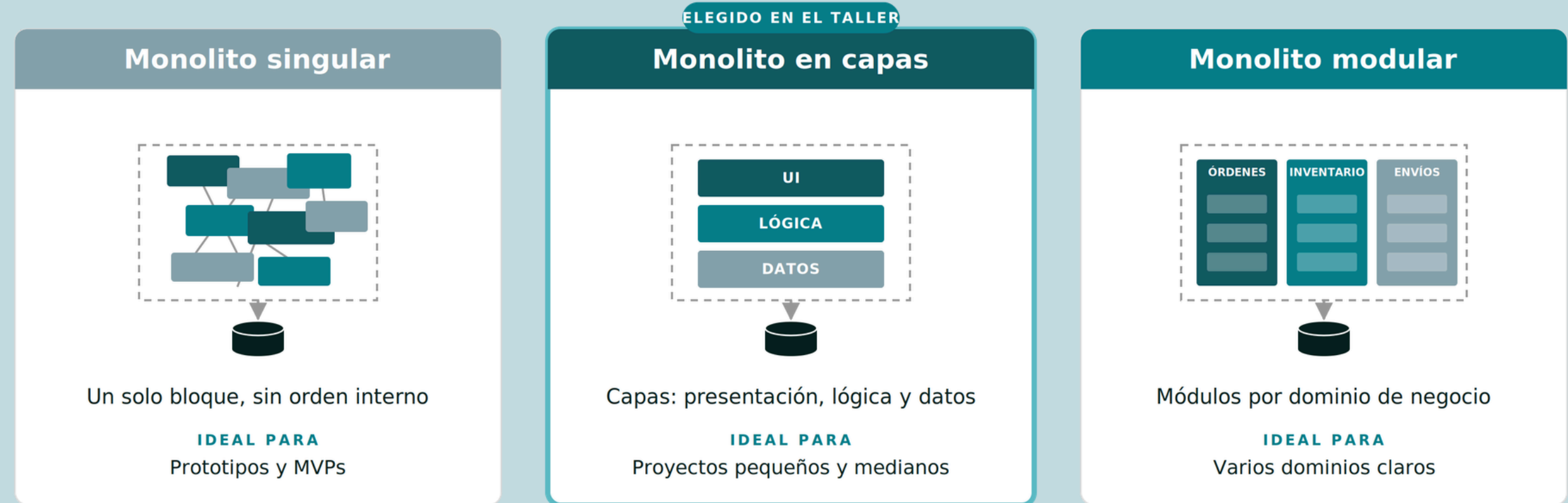
NO ES es código desordenado, no tiene organización interna y no es solo para apps pequeñas.



ESTILO ARQUITECTÓNICO: MONOLITO

Características

- Base de código única
- Almacenamiento centralizado
- Despliegue unificado
- Comunicación síncrona entre componentes
- Espacio de memoria compartido



Ventajas

Despliegue fácil, alto rendimiento y pruebas más simples.

Desventajas

Cualquier cambio obliga a redesplegar todo, escala solo como bloque y es difícil para equipos grandes.

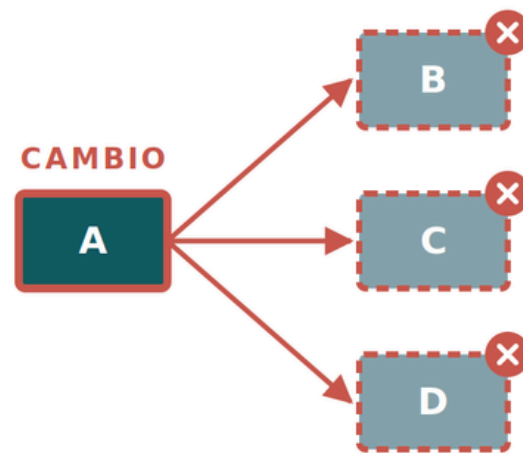
ESTILO ARQUITECTÓNICO: MONOLITO

PROBLEMAS COMUNES



Big Ball of Mud

Todo se conecta con todo,
sin estructura ni control



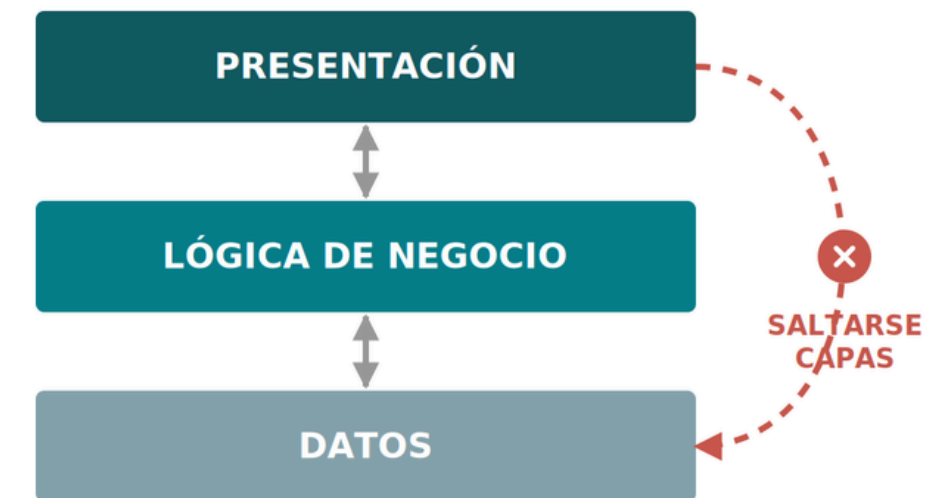
Acoplamiento excesivo

Un cambio pequeño afecta
módulos que no debería

SE EVITA
CON



PATRÓN: ARQUITECTURA EN CAPAS



Cada capa solo habla con su vecina

Presentación no accede directo a datos:
pasa siempre por la lógica de negocio

Cada componente solo puede interactuar con la capa que tiene inmediatamente arriba o abajo, una capa le presta servicios a la capa superior y le pide servicios a la capa inferior.

ESTILO ARQUITECTÓNICO: MONOLITO



Proyectos pequeños o medianos

Equipos pequeños

Construir rápido y fácil de operar



Escalar cada componente por separado

Equipos muy grandes en paralelos

Negocio con partes muy distintas que se actualizan a parte



Monolito Modular

30 TB+

Datos por minuto

32 M+

Solicitudes por minuto en Black Friday



Monolito

9

Servidores

Redis - Caché

Elasticsearch - búsquedas

STACK ASIGNADO: VISIÓN GENERAL

STACK TECNOLÓGICO: VISIÓN GENERAL



Un solo despliegue, un solo proceso

Las capas se llaman directo, sin red (no es REST ni gRPC)

STACK ASIGNADO: FRONT

JavaServer Faces

Arma pantallas con piezas ya hechas

Ventaja: Separa visual y lógica, trae muchos componentes.

Desventaja: Difícil de aprender, menos usado.

✓ Java EE o mantener sistema antiguo

✗ Proyectos nuevos

FedEx



WebSphere

Thymeleaf

Arma páginas HTML insertando datos del backend.

Ventaja: fácil de leer - HTML

Desventaja: escritura de código dentro confusa

✓ Proyectos en Spring Boot o simples

✗ Interfaces muy interactivas



JHipster

STACK ASIGNADO: FRONT

¿Relación con estilo?

Los dos encajan porque las pantallas viven en la misma aplicación que la lógica.

No necesitan red ni servicios externos: llaman directo a las funciones del backend.

¿Qué tan común?

Usar los dos a la vez no es común, porque hacen lo mismo.

Se escoge uno: JSF si el proyecto es antiguo, Thymeleaf si es nuevo con Spring Boot.

STACK ASIGNADO: BACK

Spring Boot/Java

Framework, facilita la creación de aplicaciones

- Configuración automática
- Servidor embebido (Tomcat)
- Creación rápida con Spring Initializr
- Menos código repetitivo

Desventaja: Dependencias innecesarias.

- ✓ Desarrollo rápido
- ✗ Control detallado de configuración



¿Relación con estilo?

Permite tener toda la aplicación corriendo dentro de un mismo proceso sin servicios externos.

¿Qué tan común?

Mucho

STACK ASIGNADO: BASE DE DATOS

Base de Datos: PostgreSQL/SQL

Base de datos relacional organiza info. en tablas.

- Modelo relacional
- Transacciones ACID
- Código abierto y gratuito
- Soporta JSON y personalización

Desventaja: Administración más compleja, lento en consultas muy simples.

- ✓ Datos relacionados y confiables.
- ✗ se requieren datos simples sin relaciones.



¿Relación con estilo?

PostgreSQL almacena los datos y Spring Boot accede a ellos dentro del mismo despliegue.

¿Qué tan común?

Mucho

ATRIBUTOS DE CALIDAD

Adecuación funcional

NEUTRAL

Depende de cómo se implemente la lógica

Eficiencia de desempeño

ALTO

Llamadas directas, sin latencia de red

Compatibilidad

BAJO / MEDIO

Cuesta integrarse con otros sistemas

Capacidad de interacción

NEUTRAL

Depende de la capa de presentación

Confiabilidad

BAJO / MEDIO

Un fallo puede afectar todo el sistema

Seguridad

MEDIO / ALTO

Un solo punto de despliegue y de control

Mantenibilidad

MEDIO

Depende de respetar las capas

Portabilidad

MEDIO / ALTO

Un solo paquete, fácil de mover

Seguridad operacional

NEUTRAL

Depende de los controles implementados

☒ Alto / Medio-Alto: el estilo lo favorece ☐ Medio: depende del orden interno ☐ Bajo / Medio: el estilo lo limita ☐ Neutral: no lo define el estilo

PRINCIPIOS SOLID

Single Responsibility -Neutral- lo facilita al permitir capas bien delimitadas.

Open-Close -Medio- tiende a modificar directamente las clases no extenderlas

Liskov Substitution -Neutral- es igual de aplicable que en cualquier otro estilo.

Interface Segregation -Alto- Se respeta fácilmente definiendo interfaces específicas por capa.

Dependenci inversion -Alto- Spring Boot lo favorece por inyección de dependencias

PRINCIPIOS (OTROS)

KISS -Alto- un solo despliegue, sin redes ni protocolos extra.

DRY -Alto- Una sola base de código, lógica reutilizable.

YAGNI -Alto- Evita construir infraestructura que no necesita

Composición sobre Herencia -Neutral- No depende del estilo, sino del diseño de clases.

PoLA -Alto- Cumple si las capas siguen convenciones claras (Controller → Service → Repository)

Ley de Demeter -Medio- Sin buenas practicas se accederia directamente a objetos de una capa en vez de pasar por interfaz.

STUPID -Medio- Riesgo de acoplamiento fuerte y duplicación .

ANÁLISIS DE MERCADO LABORAL

Estilo y Stack	Java	Spring Boot	Thymeleaf	JSF	PostgreSQL	Complementos (Maven, Docker, JPA/Hibernate)
Salario	\$160.000 USD/año	\$124.000 USD/año	Parte del rol Java/Spring	\$77.000 GBP/Año	\$123.000 USD/año	Dependiendo del rol
Estado actual	Muy Utilizado	Muy Utilizado	Utilizado en ocasiones	Bajo Uso	Muy Utilizado	Muy Utilizados
Demanda Laboral	Estable	Creciente	Estable	Descendiente	Muy Alta	Alta

DIAGRAMA DE ALTO NIVEL (HLD)

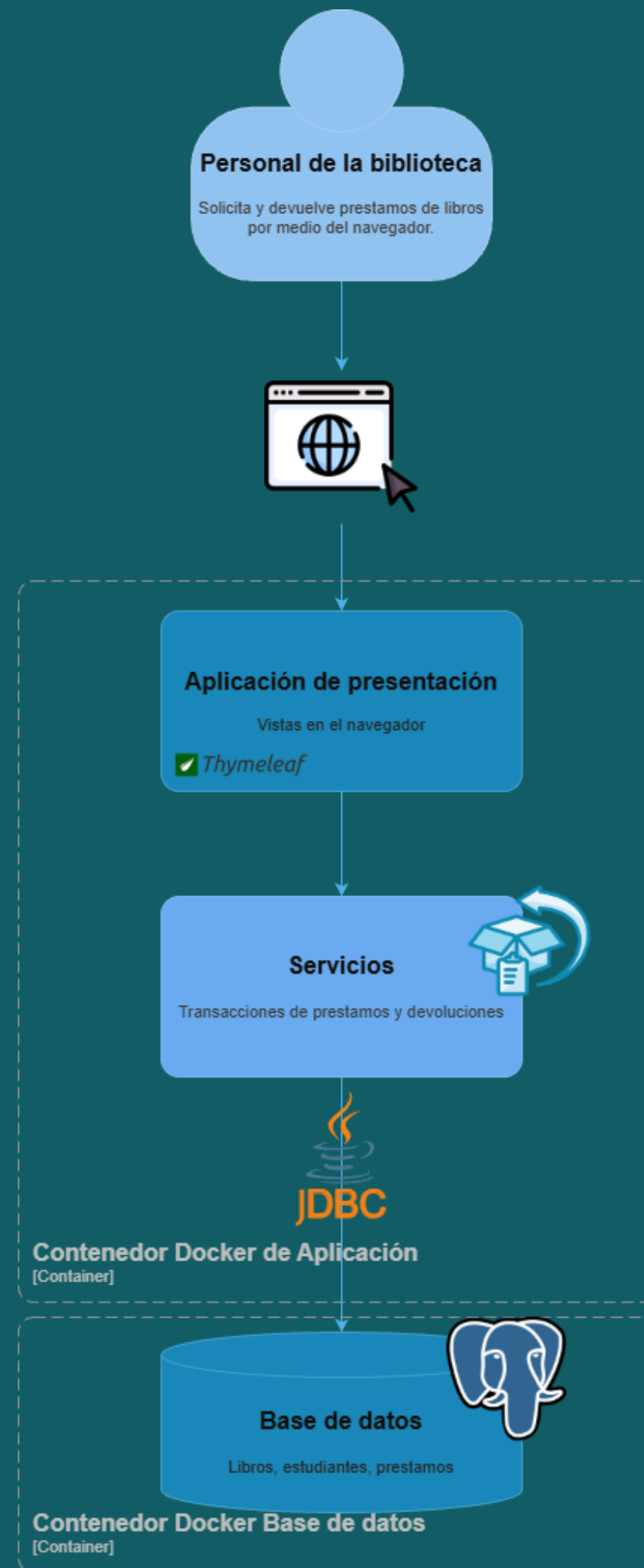


DIAGRAMA DE CONTEXTO C1

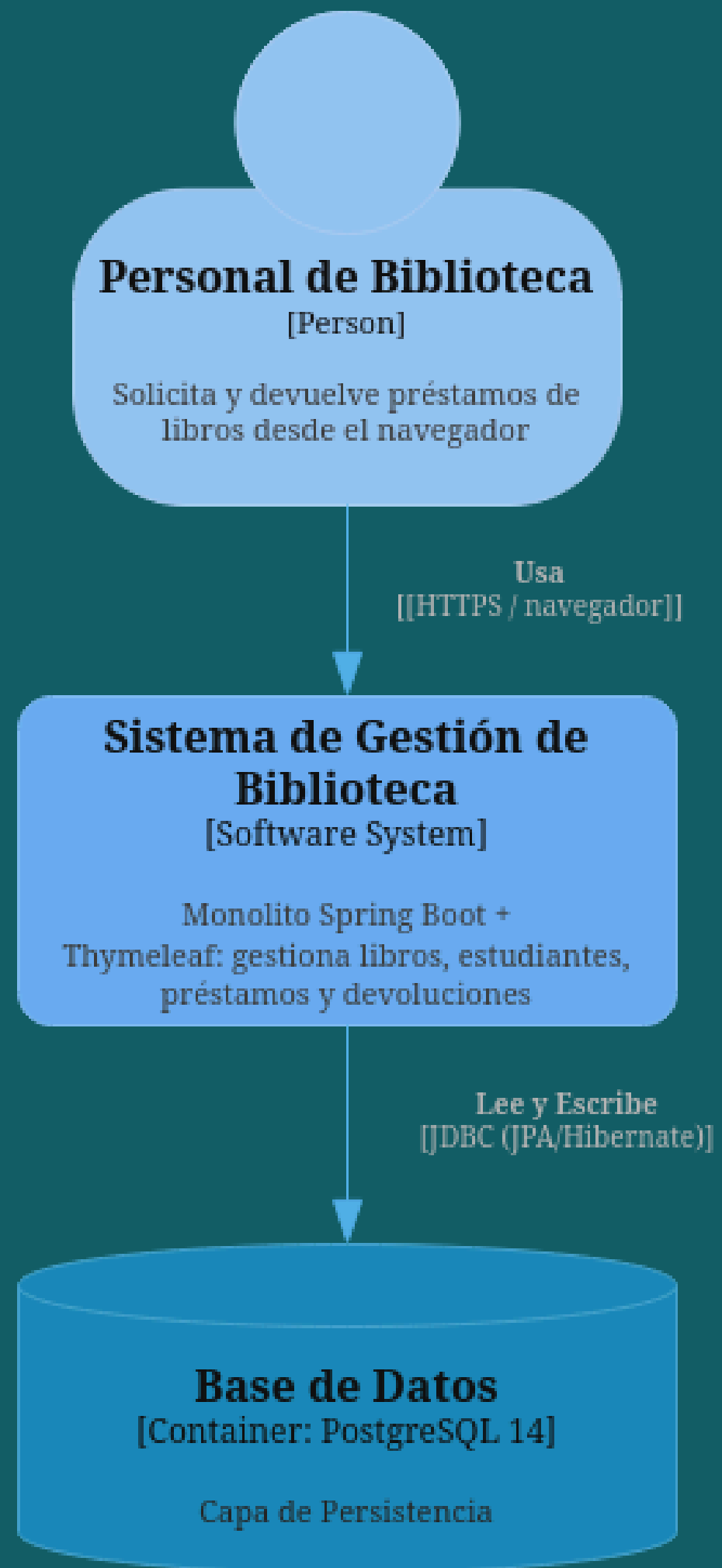


DIAGRAMA DE CONTENEDORES C2

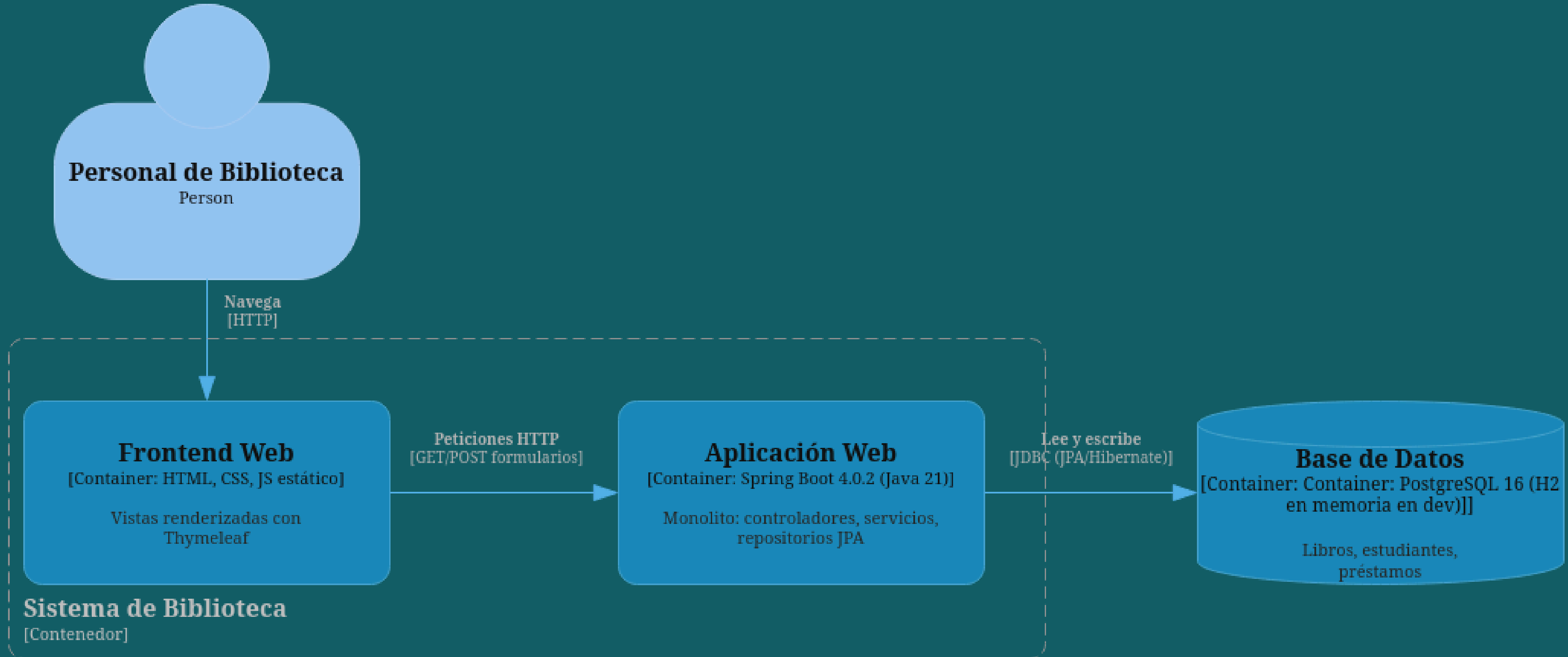


DIAGRAMA DINAMICO

PRESTAMO Y DEVOLUCIÓN

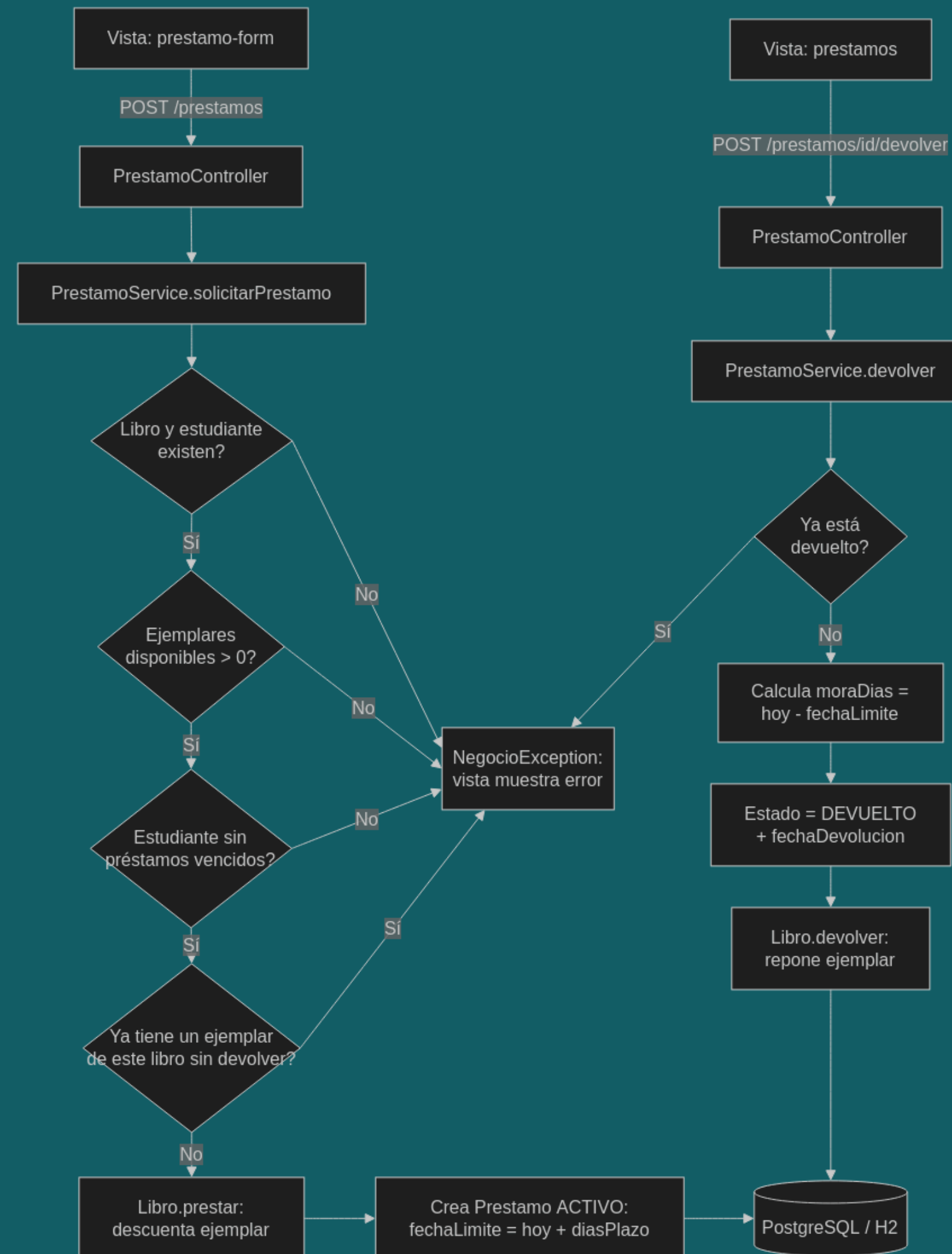
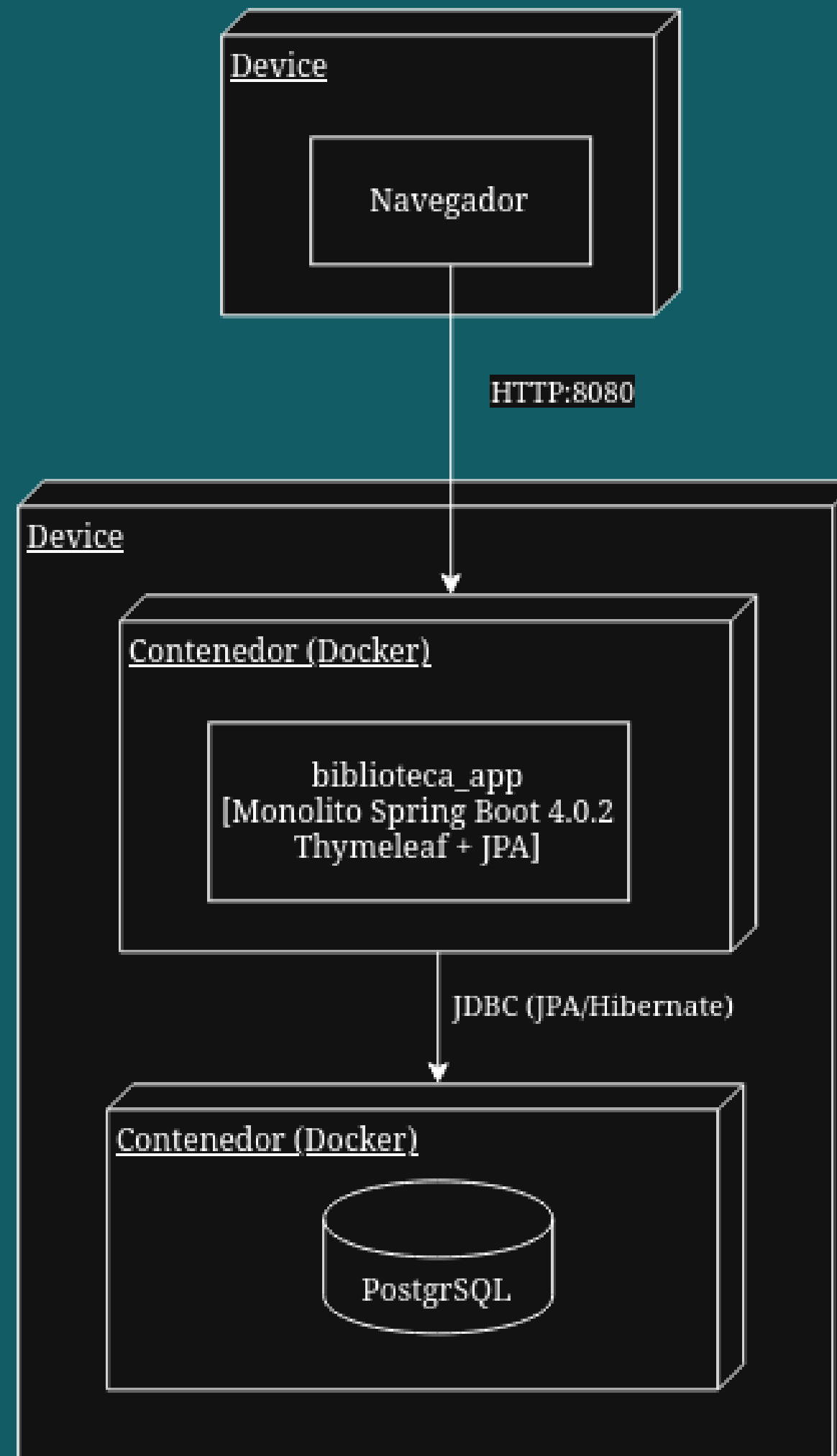


DIAGRAMA DE DESPLIEGUE



STACK ASIGNADO: FRONT

Conceptos



Aclarar

Recordar

Ejemplos.

Técnica



Diagramas

Atributos

Stack

Monolito



Orden

Balance

Contexto

GRACIAS